

ShopValue – Comprehensive Product & Go-To-Market Blueprint (90-Day Plan)

Owner: You (senior dev, 9y) \ **Stack:** Next.js + Vercel (primary), Supabase/Postgres (or Neon), Stripe, n8n \ **Mission:** Turn the Flip.ro price tracker into a monetized, low-maintenance SaaS with complementary affiliate revenue and clear expansion paths.

1) Executive Summary

What ShopValue is: a Romanian price history & alerting tool focused first on Flip.ro. It tracks historical prices, visualizes real trends (not fake discounts), and alerts users when items hit personal target prices.

\ **Value props:**

- **Truth over hype:** Black Friday reality checks (clear history, median baselines, and real “lowest in X days”).
- **Never overpay:** Personal watchlists + alerts via email/Telegram when products drop below targets.
- **Lightweight & fast:** Simple UI, zero clutter, honest insights.

90-day goals:

- **MRR:** €1,000–€2,000 from Pro subscriptions + early merchant plans.
- **Users:** 1,000–3,000 registered, 150–300 Pro conversions.
- **Affiliate:** First affiliate payouts; 3–8% of traffic monetized via outbound links.

North-star metrics: Pro trials → paid conversion, active watchlists, alert sends and CTR, weekly retained users.

2) Current State (from your description)

- Working Flip.ro scraper + historical storage.
- Product pages with current/min/max/avg and price history chart.
- Early browsing experience (products grid + product detail pages).

Gaps to close for SaaS: Auth + billing + watchlists/alerts; SEO + shareable charts; affiliate links; compliance docs; observability; weekly digest loop.

3) Product Vision & Positioning

- **Audience v1:** Romanian consumers who buy refurbished/discounted devices on Flip.ro.
- **Audience v2:** Small merchants/power users who want competitor monitoring & weekly deltas.
- **Promise:** “Vezi prețul real în timp și primește alerte când merită să cumperi.”
- **Hook feature: Black Friday Score (BFS):** quantifies how “real” a discount is vs recent medians.

Pricing (start here):

- **Free:** 2 alerts, 30-day history.
 - **Pro €4.99/mo (or €39/yr):** unlimited alerts, 180–365-day history, CSV export, email + Telegram, watchlists up to 50 items.
 - **Merchant €59/mo (later):** 250 SKUs, competitor watchlists, weekly CSV/PDF, email digests.
-

4) Roadmap (Weeks 1–6 build, 7–12 grow)

Weeks 1–2 (Monetize core):

- Auth (magic link), Stripe billing, watchlists & alerts (email + Telegram), weekly digest, basic analytics.
- BFS badge & PDP improvements (shareable OpenGraph charts).

Weeks 3–4 (Acquisition & revenue):

- Affiliate link wrapping (2Performant / Profitshare or similar).
- SEO foundations: sitemap, JSON-LD, canonical, OG image endpoint.
- Crawl hardening + on-demand refresh on PDP views.

Weeks 5–6 (Polish & Merchant beta):

- Dashboard polish, CSV export, creator landing, basic merchant watchlists.
- Press/creator outreach pack; early merchant trials.

Weeks 7–12 (Scale):

- Growth loops (creator partnerships, BF content).
 - Ops automation in n8n (alerts, billing webhooks, digests, ops digest).
 - Churn defense, “pause plan”, yearly plan promotion.
-

5) System Architecture (Reference)

Frontend: Next.js (App Router), ISR for PDPs, Edge functions for OG images and lightweight APIs.

Backend: Next.js API routes or a tiny Fastify/Express service for scraping callbacks. **DB:** Postgres (Supabase/Neon). Optional TimescaleDB or partitioned tables for price points. **Background jobs:** n8n (cron, webhooks) + serverless cron (Vercel Cron or external). **Email:** Resend/SendGrid. **Messaging:** Telegram bot for alerts. **Payments:** Stripe (Billing + Customer Portal). **File/Images:** Cloudinary/S3 for chart snapshots if needed.

Key Services

- **Crawling workers:** Playwright/undici fetchers, rotating UA, cache, rate limit; write to `price_points`.
 - **Alert engine (n8n):** join latest `price_points` to user `watchlists`, evaluate rules, send alerts.
-

6) Data Model (Prisma Sketch)

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  stripeCustomerId String? @unique
  plan        Plan      @default(FREE)
  createdAt   DateTime  @default(now())
  watchlists  Watchlist[]
  events      Event[]
}

enum Plan { FREE PRO MERCHANT }

model Product {
  id          String    @id @default(cuid())
  slug        String    @unique
  title       String
  brand       String?
  category    String?
  createdAt   DateTime  @default(now())
  variants    Variant[]
}

model Variant {
  id          String    @id @default(cuid())
  productId   String
  attrsJson   Json      // condition, storage, color, etc.
  product     Product   @relation(fields: [productId], references: [id])
  pricePoints PricePoint[]
}

model PricePoint {
  id          String    @id @default(cuid())
  variantId   String
  ts          DateTime  @default(now()) @index
  price       Int       // in RON (minor units optional)
  source      String    // flip.ro + path
  available   Boolean   @default(true)
  variant     Variant   @relation(fields: [variantId], references: [id])
}

model Watchlist {
  id          String    @id @default(cuid())
  userId      String
  variantId   String
  targetPrice Int
  mode        WatchMode @default(BELOW) // BELOW or AT_OR_BELOW
  createdAt   DateTime  @default(now())
  user        User      @relation(fields: [userId], references: [id])
}
```

```

    variant      Variant    @relation(fields: [variantId], references: [id])
  }

  enum WatchMode { BELOW AT_OR_BELOW }

  model Event {
    id          String    @id @default(cuid())
    userId      String?
    type        String
    payload      Json
    ts          DateTime @default(now())
    user        User?     @relation(fields: [userId], references: [id])
  }

  model Subscription {
    id          String    @id @default(cuid())
    userId      String    @unique
    stripeSubId String    @unique
    plan        Plan
    status       String
    currentPeriodEnd DateTime
    user        User     @relation(fields: [userId], references: [id])
  }

  model AffiliateClick {
    id          String    @id @default(cuid())
    productId   String
    variantId   String?
    userId      String?
    url         String
    ts          DateTime @default(now())
  }

  model CrawlJob {
    id          String    @id @default(cuid())
    variantId   String
    status       String    // queued, running, success, failed
    runAt       DateTime
    message     String?
  }

```

Indexes you'll want: `(variantId, ts)`, `(userId, variantId)` on Watchlist, and partial index on last 30/90 days for fast aggregates.

7) Core Feature Specs

7.1 Watchlists & Alerts

- **Add to watchlist** on PDP → modal with target slider seeded at **90-day median -10%**.

- **Constraints:** Free max 2 alerts; Pro unlimited.
- **Alert channels:** email (default), Telegram (optional).
- **Alert logic:** Trigger if `current_price ≤ targetPrice` (mode BELOW) and **we haven't alerted for this rule in last 48h**.

n8n: `` (hourly)

- Cron → Postgres(SELECT latest price_points last hour) → Join with watchlists where target ≤ price → Function(build template) → HTTP Request(Resend/SendGrid) → HTTP Request(Telegram) → INSERT Event.

7.2 Black Friday Score (BFS)

For variant **v** at time **t**:

- `median90 = median(price over last 90 days where available=true)`
- `lowest180 = min(price over last 180 days where available=true)`
- `realDiscount = 1 - (current / median90)`
- `allTimeFactor = 1 - (current / lowest180)`
- `BFS = clamp( 0.7*realDiscount + 0.3*allTimeFactor , 0, 1 )`
- Display as **score 0–100** with labels: Poor (0–29), Meh (30–59), Real Deal (60–79), Great (80–100).
- Show **badges**: “Cel mai mic preț în 180 zile” if `current == lowest180`.

7.3 Shareable Charts & SEO

- **OG image endpoint** `/og/product/[id]/chart.png` → render last 180-day chart server-side.
- **JSON-LD Product + Offer** on PDPs.
- **Sitemap** weekly, includes `lastmod` based on last price change.
- **Canonical** PDP URL to avoid duplicate variants.

7.4 Affiliate Integration

- Wrap outbound “Cumpără acum” with affiliate network links (where applicable).
- Server-side redirect endpoint `/go/[product]/[variant]` records `AffiliateClick` then 302 to partner URL with tags.
- UTM propagation: preserve `utm_source=shopvalue` etc.

7.5 Merchant Beta (Weeks 5–6)

- Allow adding competitor SKUs by URL; crawl nightly; weekly email: current price, delta 7/30 days, min/max history snapshot.
- Export CSV with columns: `sku, current, min90, max90, median90, delta7, delta30`.

8) API Endpoints (sketch)

- GET `/api/products/:slug` – PDP data (product, variants, recent prices, aggregates).
- GET `/api/variants/:id/history?days=180` – timeseries.
- POST `/api/watchlists` – {variantId, targetPrice, mode}.
- POST `/api/alerts/test` – triggers a test alert to verify channel.
- POST `/api/stripe/webhook` – billing updates.

- GET /go/:product/:variant – affiliate redirect & tracking.
 - GET /og/product/:id.png – server-rendered chart.
-

9) UI/UX Notes

- PDP top: current price, **BFS**, lowest180 badge, big **Add to watchlist** button.
 - Chart: area/line with tooltips; toggle **30/90/180 days**.
 - Copy tone (RO): “Nu plăti mai mult decât merită. Vezi istoricul și setează o alertă.”
-

10) Growth Loops & GTM

10.1 Launch Loop (next 2–3 weeks)

- Landing: “**Scorul Black Friday 2025** – vezi cât de reală e reducerea.” Input: product URL → BFS + chart → **Ask email to save alert**.
- Post evidence in RO tech/deals communities (screen + share link).
- Add “**Share**” buttons that copy PDP URL with OG image preview.

10.2 Creator Loop

- Offer creators **Pro free** + “open PDP with BFS badge” links.
- Track creator referrals via unique UTM → monthly payout or shoutouts.

10.3 SEO Loop

- PDP titles: Istoric preț {{model}} – min/max, medie, alerte ({{luna}} {{anul}})
 - 4 evergreen blog posts with embedded charts: “Evoluția prețurilor iPhone/Pixel”, “Mituri Black Friday”.
-

11) Operations, Compliance, and Observability

- **GDPR**: consent for emails, clear unsubscribe; Privacy & Terms pages; “Not affiliated” disclaimer.
 - **Cookies**: implement consent banner; only functional by default.
 - **Uptime & Errors**: Vercel + Sentry; alert on crawler failures; health checks for cron.
 - **Backups**: daily Postgres snapshots; object storage for HTML cache (if stored).
 - **Rate limits**: IP & per-user for endpoints, bot protection on forms.
-

12) Analytics & KPIs

- **Acquisition**: signups/day, landing→signup conversion, creator referrals.
 - **Engagement**: weekly active, watchlists per user, alerts sent, CTR.
 - **Monetization**: trials, conversion to Pro, ARPU, affiliate clicks.
 - **Retention**: D7/D30 retention, churn, paused plans.
-

13) n8n Automations (node-by-node)

13.1 alert-dispatcher

- Cron (*/*30) → Postgres (SELECT latest price_points in window) → Function (match vs watchlists) → HTTP Request (Resend email) → HTTP Request (Telegram bot) → Postgres (INSERT Event rows) → Deduplicate (don't re-alert within 48h).

13.2 billing-webhooks

- Webhook /api/stripe/webhook → Verify signature → Switch (event type) → Upsert Subscription + User.plan → Email "Pro activ".

13.3 weekly-digest

- Cron (Mon 09:00) → Postgres (aggregate watchlists with price drops last 7d) → Loop users → Email digest.

13.4 ops-digest

- Cron (daily 08:00) → Postgres (signups, alerts sent, failures) → Email/Slack message.

14) Implementation Checklists

Week 1

-

Week 2

-

Week 3

-

Week 4-6

-

15) Sample Code Snippets

15.1 Next.js (route handler: create watchlist)

```
// app/api/watchlists/route.ts
import { auth } from '@lib/auth';
import { prisma } from '@lib/prisma';
```

```

import { NextResponse } from 'next/server';

export async function POST(req: Request) {
  const user = await auth();
  if (!user) return new NextResponse('Unauthorized', { status: 401 });

  const { variantId, targetPrice, mode } = await req.json();
  // enforce limits for Free plan
  const count = await prisma.watchlist.count({ where: { userId: user.id } });
  const plan = user.plan; // from session/db
  const limit = plan === 'FREE' ? 2 : 1000;
  if (count >= limit) return NextResponse.json({ error: 'limit' }, { status: 402 });

  const wl = await prisma.watchlist.create({ data: { userId: user.id, variantId, targetPrice, mode } });
  return NextResponse.json({ id: wl.id });
}

```

15.2 Stripe Webhook Skeleton

```

// app/api/stripe/webhook/route.ts
import Stripe from 'stripe';
import { headers } from 'next/headers';
import { prisma } from '@lib/prisma';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, { apiVersion: '2024-04-10' });

export async function POST(req: Request) {
  const sig = headers().get('stripe-signature')!;
  const buf = Buffer.from(await req.arrayBuffer());
  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(buf, sig, process.env.STRIPE_WEBHOOK_SECRET!);
  } catch (err: any) {
    return new Response(`Webhook Error: ${err.message}`, { status: 400 });
  }

  switch (event.type) {
    case 'customer.subscription.updated':
    case 'customer.subscription.created': {
      const sub = event.data.object as Stripe.Subscription;
      const customerId = sub.customer as string;
      const user = await prisma.user.findFirst({ where: { stripeCustomerId: customerId } });
      if (user) {
        await prisma.subscription.upsert({
          where: { userId: user.id },

```

```

        update: {
            stripeSubId: sub.id,
            plan: sub.items.data[0]?.price.nickname === 'pro' ? 'PRO' :
'FREE',
            status: sub.status,
            currentPeriodEnd: new Date(sub.current_period_end * 1000)
        },
        create: {
            userId: user.id,
            stripeSubId: sub.id,
            plan: 'PRO',
            status: sub.status,
            currentPeriodEnd: new Date(sub.current_period_end * 1000)
        }
    });
}
break;
}
}
return new Response('ok');
}

```

15.3 BFS Computation (SQL-ish)

```

WITH last90 AS (
    SELECT price
    FROM price_point
    WHERE variant_id = $1 AND ts >= now() - interval '90 days' AND available =
true
), last180 AS (
    SELECT price
    FROM price_point
    WHERE variant_id = $1 AND ts >= now() - interval '180 days' AND available
= true
), stats AS (
    SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY price) AS median90,
           MIN(price) AS lowest180
    FROM last90, last180
)
SELECT ROUND(100 * GREATEST(0, LEAST(1, 0.7*(1 - $2/stats.median90) + 0.3*(1
- $2/stats.lowest180))),
       stats.median90, stats.lowest180
FROM stats;

```

16) Testing & Quality

- **Unit:** BFS calc, watchlist creation limits, webhook signature verification.
- **Integration:** alert-dispatcher E2E (seed price drop → assert email).

- **Load:** PDP history API under cached & uncached scenarios.
 - **Manual:** domain flows, affiliate redirects.
-

17) Risks & Mitigations

- **Source changes / anti-bot:** Keep crawlers modular; on-demand crawl on PDP view; cache HTML; quick adapters.
 - **Legal/ToS:** use only publicly visible info; add rate limits; remove on request; disclaimers.
 - **Monetization delay:** add affiliate ASAP; seed creator partnerships to drive early paying users.
 - **Churn:** weekly digest and multi-channel alerts; “pause plan” and “set new target” prompts.
-

18) Team & Roles (make it semi-passive)

- **You:** core dev, infra, billing, BFS, SEO.
- **Friend:** crawler maintenance, data QA, creator outreach.
- **Girlfriend (AI Ops):** n8n ownership (alerts, billing, digests), support macros, help articles, social snippets; content/project QA using AI prompts.

AI prompt patterns for her

- “Summarize this PR + generate test cases for edge conditions (list).”
 - “Draft a help article for feature X in RO, friendly tone, 200–300 words.”
 - “From this SQL schema, propose 10 integrity tests and seed data.”
-

19) Budget/Time Estimates (first 6 weeks)

- Core auth/billing/watchlists/alerts: \~30–36h
 - SEO + OG + BFS: \~14–18h
 - Affiliate + on-demand crawl + rate limiting: \~16–20h
 - Emails + digests + polish + compliance pages: \~10–14h\ **Total:** \~70–88h (fits 3–5 weeks at 20h/week)
-

20) Definition of Done (Phase 1)

- Users can sign up, create watchlists, receive alerts, upgrade to Pro, cancel in portal.
 - BFS visible on PDPs; OG share shows chart; sitemap & JSON-LD valid.
 - Affiliate redirects working; at least one partner network active.
 - n8n flows stable (alerts, billing, weekly & ops digests).
 - Privacy/Terms live; error monitoring; daily DB backups.
-

21) Outreach & Messaging (RO templates)

Email:

Subiect: Vreți să știți dacă reducerea e reală? Bună, am construit ShopValue – vezi istoricul real al prețurilor pe Flip și primești alerte când chiar merită. Exemplu: {{product_link}} (BFS: {{score}}). Încearcă gratuit (2 alerte), Pro e 4,99 €/lună.

Creator DM:

Salut! Am un tool care arată **cât de reale** sunt reducerile (BFS) + alerte. Îți dau Pro gratuit și linkuri speciale pt audiența ta. Vrei un demo de 5 minute?

22) Future Expansion

- Add more marketplaces (eMAG refurb, OLX shops) with toggle per source.
 - Price predictions (ARIMA/light models) as a Pro+ feature.
 - Community lists: “Top scăderi din această săptămână”.
-

Final Notes

Keep the scope brutally small: **alerts + BFS + shareable charts + affiliate links** win first. Everything else is additive. Ship weekly, talk to users, and let the data pick your next feature.